

Dev Log — 2026-07-24 — Snake

Session Goal

Finish wiring the dynamic `Action / KeyMap` input system from a compiling-but-inert state to actually driving `Snake` movement, ending on a running, green build. Scope grew mid-session once it became clear `Snake` shouldn't own input bindings directly — it isn't an `Entity` — which led to designing and shipping a real ECS `InputComponent` on the snake's head instead.

Work Completed

- Renamed `Core::Input::ActionBindings` → `Core::Input::ActionRouter` throughout (`action-router.hpp / .cpp` , `IInputSystem::GetActionRouter()` , `IApplication::GetInputActionRouter()`), and implemented it end to end: `Bind / BindAxis` return a move-only RAI `Handle` , `Update()` polls `axisBindings` each frame via `IInputDispatcher::GetActionValue` , `Unbind` erases by handle id. This closes UE-0041.
- Moved `InputActionPressedEvent / InputActionReleasedEvent` from `Engine::Events::Input` to `Core::Events::Input` — they only ever held a `Core::Input::Action` , so nothing about them required `Engine` . This let `ActionRouter` (which must stay a `Core` type, since `IApplication::GetInputActionRouter()` returns it) subscribe to them directly without reaching into `Engine` .
- Added explicit constructors to both event structs (`InputActionPressedEvent(Action)` , `InputActionReleasedEvent(Action)`) after discovering aggregate-initialization stopped working the moment they inherited from `Core::Events::IEvent` (virtual destructor disqualifies aggregate status for the whole hierarchy, not just the base) — `input-dispatcher.cpp:42,45` 's `InputActionPressedEvent{action}` calls needed a real constructor once that stopped being an aggregate.
- Introduced `Core::Utils::Zip / Overloaded` (`core/utils/zip.hpp`) — a generic `std::visit`-based helper for pairwise-combining two same-alternative `std::variant`s — and rebuilt `Accumulate / Distance` (`action-value.hpp`) on top of it instead of hand-rolled `std::visit` calls each. Fixed two real bugs found while building it: `ValueTypeOf` tested `bool` twice instead of `bool` then `float` (UE-0039, silently misclassified every `Axis1D` value as `Boolean`), and `Distance` 's `bool` case used `||` instead of `!=` (equal `true / true` reported as different).
- Added `Vector2D::Length()` as an instance-method wrapper over the existing static `Vector2D::Magnitude(v)` , so `(x - y).Length()` reads naturally at `Distance` 's `Vector2D` call site without duplicating the `sqrt(x*x + y*y)` math.
- Built `Core::Input::Components::IInputComponent` (interface, `Core`) and `Engine::Input::Components::InputComponent` (implementation, `Engine`) — a real `Core::Components::IComponent` , addable via `Entity::AddComponent<T>` like `TransformComponent2D / ColliderComponent2D` . `Bind / BindAxis` forward straight to an attached

`ActionRouter` ; there's no pending-queue — the attach vs. bind ordering is resolved by lifecycle hook placement instead (see Architectural Decisions).

- Added `Engine::Entities::Registrars::InputComponentRegistrar` (`TypedComponentRegistrar<IInputComponent>`), registered into `EntityComponentPipeline` next to the existing `RenderComponentRegistrar`. Its `OnRegistered` calls `component->AttachRouter(router)` during `ComponentDispatcher::Dispatch`. `EntityComponentPipeline` and `EntityManager` both grew a `Core::Input::ActionRouter&` constructor parameter to carry it down from `GameplayScene`.
- Added `SnakeGame::SnakeHead : SnakeSegment` (`snake-head.hpp / .cpp`) — the only segment that gets an `IInputComponent`. `SnakeSegment::OnRegistration` lost `final` / became overridable to allow this. `SnakeHead::OnRegistration()` adds the component; `SnakeHead::BeginPlay()` binds `"MoveHorizontal" / "MoveVertical"` to `this->direction.x / .y`. `Snake::CreateBody()`'s first iteration now constructs a `SnakeHead` instead of a plain `SnakeSegment`, and `Snake::head` is typed `SnakeHead*`.
- `Snake::Update()` now actually applies input to movement — `SetDirection(this->head->direction)` when either axis is non-zero — closing UE-0044 (direction was never being updated from input at all going into this session). `Snake::Move()` dropped the two dead lines that queried `this->inputSystem->GetActionValue(...)` against `this->inputActions` — neither member exists on `Snake` anymore post-redesign; this also closes UE-0042/UE-0043, both superseded exactly as predicted in the last log.
- Threaded `ActionRouter` from `Application` down to where it's actually needed: `Application` now holds the concrete `std::shared_ptr<Core::Input::IInputSystem> inputSystem` (previously discarded as a local after `RegisterSystem`), deliberately declared *before* `sceneManager` in `application.hpp` so it outlives scene teardown (see Architectural Decisions). `GetInputActionRouter()` forwards to `inputSystem->GetActionRouter()`. `GameplayScene` gained `Core::Input::ActionRouter& inputActionRouter` and passes it into its `EntityManager`. `SnakeParams / Snake` no longer reference `ActionRouter / ActionSet / IInputSystem` at all — that knowledge lives entirely in `SnakeHead` and the registrar now.
- Fixed the actual root cause behind a run of `ActionSet::Get: unregistered action name` asserts: `SnakeApplication::Configure()`'s `auto inputActions = this->GetInputActionSet();` was copying the `ActionSet` by value (`GetInputActionSet()` returns a reference) — every `RegisterAction` call was mutating a throwaway copy, leaving the real `ActionSet` permanently empty regardless of what names anything used. Changed to `auto&`.
- Fixed a `Log::GetCoreLogger()` null- `unique_ptr` dereference on startup: `Application::Run()` called `Configure()` before `RegisterDependencies() / Initialize()` (the two places `LOG_INIT` lived), but `ActionSet::RegisterAction / BuildKeyMap` (called from `Configure()`) now log via `LOG_CORE_INFO`. Moved a `LOG_INIT()` call to the top of `Run()`, before `Configure()`.
- Cleared four separate link failures on the way to green: `action-router.cpp`'s stale `#include "action-bindings.hpp"` (file no longer existed), `engine/input/components/input-component.cpp` and the two new event `.cpp` files missing from `CMakeLists.txt`'s `add_executable(...)` list, and `SnakeHead::SnakeHead(const SnakeHeadParams&)` declared but never defined.
- Reconciled two `RegisterAction / Bind(...)` / `BindAxis(...)` action-name mismatches (`"MoveForward" / "MoveRight"` vs. `"MoveVertical" / "MoveHorizontal"`) across `Configure()` and `SnakeHead::BeginPlay()` — settled on `MoveVertical / MoveHorizontal` everywhere.

Lessons Learned

- A class becomes ineligible for aggregate initialization the moment *any* virtual function exists anywhere in its hierarchy — including an implicitly-declared destructor that's virtual only because it overrides a virtual base destructor. Confirmed with a minimal repro (`struct Base { virtual ~Base()=default; }; struct Derived : Base { Action a; }; — Derived{a} fails to compile) before trusting the diagnosis.`
- Compared this session's `ActionRouter` / `InputComponent` /digital-vs-axis split against Unreal's input model throughout: `UInputComponent::BindAxis(name, this, &Fn)` 's two-argument (object pointer + member-function-pointer) delegate shape is what a capturing lambda (`[this](ActionValue v){...}`) replaces in one piece in modern C++ — no need for Unreal's separate `Object` / `Func` split, since the closure already carries both. Also mapped `Bind` / `InputActionPressedEvent` onto Unreal's `IE_Pressed` / `IE_Released` action bindings, and `BindAxis` / `ActionRouter::Update()` polling onto Unreal's per-tick `GetActionValue` axis reads.
- Considered moving `ActionRouter` 's subscribe/poll responsibilities directly into each `InputComponent` (so components self-route instead of delegating to one shared object) as a way to remove the "attach dependency after construction" awkwardness. Concluded it doesn't remove the attach step (still needs `eventBus` / `actionSet` / `dispatcher` from outside) and trades one shared event-bus subscription + one poll loop for N of each — more new machinery for no behavioral gain. Kept `ActionRouter` centralized; see Architectural Decisions for what actually fixed the ordering issue instead.
- A vector's length/magnitude is just Pythagoras applied to its components treated as a point's distance from the origin — $\text{sqrt}(x^2 + y^2)$, same shape as point-to-point distance with one point fixed at zero.

Architectural Decisions

- `ActionRouter` stays in `Core` , decoupled from `Engine` via two seams, rather than moving to `Engine::Input` . `IApplication::GetInputActionRouter()` (a `Core` interface) forced the choice: since it has to name the return type, `ActionRouter` can't live in `Engine` without `Core` referencing `Engine` . Resolved by (1) relocating the pressed/released events into `Core::Events::Input` — they never needed `Engine` in the first place — and (2) `ActionRouter` depending on `Core::Input::IInputDispatcher` (an existing interface) rather than the concrete `Engine::Input::InputDispatcher` .
- **Input reaches game entities through the ECS component pipeline (`InputComponentRegistrar`), not through constructor parameters threaded down `SnakeParams` / `SnakeHeadParams` .** Considered passing `ActionRouter&` explicitly into `Snake` / `SnakeHead` construction (simpler, but the user explicitly didn't want game code plumbing a dependency it never directly uses) against injecting it via a registrar mirroring the existing `RenderComponentRegistrar` pattern (`EntityComponentPipeline` already solves exactly this problem for render components). Chose the registrar — `GameplayScene` is now the only game-layer code that touches `ActionRouter` at all, to hand it to `EntityManager` ; `Snake` / `SnakeHead` never reference it.
- **`InputComponent::Bind` / `BindAxis` calls happen in `SnakeHead::BeginPlay()` , not `OnRegistration()` , with no pending-bind queue.** `EntityManager::AddEntity` runs

`componentPipeline.Run()` (which fires `OnRegistration()` then `Dispatch()`, where the registrar attaches the router) strictly before `activationPipeline.Run()` (which fires `BeginPlay()`), in the same synchronous call. Placing binds in `BeginPlay()` guarantees the router is already attached, so `InputComponent` doesn't need queue-and-flush machinery for calls that arrive before their dependency exists — confirmed against `entity-manager.cpp`'s actual call order rather than assumed.

- **SnakeHead : SnakeSegment is a real subtype, not an `index == 0` branch on a single `SnakeSegment` type.** `SnakeSegment` already leaned on an `index == 0` convention for `DebugUpdate()`, which would've been the smaller diff. Chose the subtype instead so only the head is structurally capable of holding an `InputComponent` — `SnakeSegment::OnRegistration()` had to stop being `final` to allow it.
- **Application holds the concrete `IInputSystem` as a member, declared before `sceneManager`.** Previously `Application::Initialize()` built `InputSystem` as a local, handed it to `RegisterSystem`, and let the local go out of scope — leaving `GetInputActionRouter()` nothing to forward to. Fixing that reopened a lifetime question: `Application`'s members destruct in reverse declaration order, and `sceneManager` owns every `GameplayScene` / `Snake` / `InputComponent`, whose `Handle` destructors call back into `ActionRouter` during teardown. `inputSystem` had to be declared *before* `sceneManager` so it's still alive when scene teardown runs, not after.

Known Issues Carried Forward

- **UE-0004** — BUG: Window-side event bus stub. Unchanged — `WindowResizedEvent` still an empty struct with no publisher.
- **UE-0045** — REAPER: `src/core/input/action-binding.cpp` is a dead duplicate of `action-router.cpp`. Both files have byte-identical content (a leftover from the `ActionBindings` → `ActionRouter` rename). `action-binding.cpp` isn't in `CMakeLists.txt` so it doesn't affect the build, but it's dead weight in the tree and a trap for anyone who edits `action-router.cpp` and doesn't know its twin exists. Delete it.
- **UE-0046** — BUG: `InputComponent::router` is never initialized to `nullptr`. `engine/input/components/input-component.hpp`'s `Core::Input::ActionRouter* router;` has no default member initializer, and `InputComponent::InputComponent() {}` doesn't set it either — it holds an indeterminate value until `AttachRouter` runs. In practice this is unreachable today because `BeginPlay()` always runs after `AttachRouter` (see this session's Architectural Decisions), but the `assert(this->router && ...)` guards in `Bind` / `BindAxis` are reading that indeterminate value if that ordering assumption is ever violated, which is itself UB before the assert even gets a chance to fire cleanly. Add `= nullptr` to the declaration.
- **UE-0047** — REAPER: small leftovers from this session's rename churn. Two harmless but worth-sweeping items: `input-component.hpp` forward-declares an unused, misspelled `namespace Core::Input { class ActionRouters; }` (plural, never referenced — `action-router.hpp` is `#include`d directly instead); `snake.cpp`'s `Snake::Snake` constructor initializer list still has a commented-out `// , inputActions(snakeParams.inputActions)` line from before `SnakeParams` dropped that member.

Next Session

- ■ **UE-0038** — MAIN QUEST: Migrate `Confirm / Decline` off the dead `InputSystem::IsActionPressed` API. Now directly actionable — `Confirm / Decline` are registered as `Boolean` actions and key-bound (`Enter / E` , `Escape`) in `Configure()` , and `ActionRouter::Bind(name, callback)` is fully implemented and proven working (it's what `SnakeHead` uses for axes, just via `BindAxis`). Nothing currently calls `ActionRouter::Bind("Confirm", ...)` / `Bind("Decline", ...)` anywhere — `main-menu-state.cpp:31` / `game-over-state.cpp:32` still reference the old dead API. Whatever owns those states needs its own `InputComponent` (or equivalent `Bind` call) the same way `SnakeHead` now does for axes.
- ■ **UE-0035** — MAIN QUEST: `GameContext::input` is never assigned; reconcile the `MainMenuState` workaround. Still blocked on UE-0038 landing.
- ■ **UE-0009** — MAIN QUEST: Implement `AppleSpawner` . Still not implemented.